

Lab1:

Data Output

Messages

Notifications

Showing rows: 1 to 13

Page No

	QUERY PLAN
	text
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.178..2.008 rows=32 loops=1)
2	Buffers: shared hit=1 read=152
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.021..0.023 rows=1 loo...
4	Index Cond: (customer_id = 1)
5	Buffers: shared hit=1 read=2
6	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.155..1.978 rows=32 loops=1)
7	Filter: (customer_id = 1)
8	Rows Removed by Filter: 16012
9	Buffers: shared read=150
10	Planning:
11	Buffers: shared hit=72 read=5 dirtied=2
12	Planning Time: 3.795 ms
13	Execution Time: 2.036 ms

Total rows: 13

Query complete 00:00:00.068

1. Identify the Roles:

Nhìn vào ảnh, bảng xuất hiện đầu tiên ở nhánh trên là bảng customer. Do đó, customer chính là Outer Table

Bảng nằm ở nhánh dưới là bảng rental. Do đó, rental chính là Inner Table

2. Why Nested Loop?

Nhìn vào Dòng số 3 (của bảng customer), ta thấy thông số trả về là **rows=1**. Tức là bảng Outer chỉ xuất ra đúng 1 dòng dữ liệu.

Lý do Hash Join quá đắt: Khi bảng Outer chỉ có 1 dòng, việc dùng Hash Join sẽ bắt hệ thống phải cấp phát bộ nhớ và chạy thuật toán băm toàn bộ dữ liệu, tạo ra chi phí khởi tạo vô ích. Dùng Nested Loop để cầm trực tiếp 1 dòng này đi quét bảng rental là phương án rẻ và nhanh nhất.

3. Efficiency:

Nhìn vào Dòng số 6, ta thấy thông số loops=1.

Giải thích: Số loops của bảng Inner phụ thuộc vào số lượng dòng mà bảng Outer ném xuống. Vì bảng customer ở trên chỉ ném xuống đúng 1 khách hàng (rows=1), nên hệ thống chỉ cần kích hoạt vòng lặp quét bảng rental đúng 1 lần duy nhất.

Lab2:

QUERY PLAN	
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.194..0.478 rows=1000 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=88 read=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.014..0.155 rows=1000 loo...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.159..0.159 rows=6 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.153..0.154 rows=6 loo...
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=158 read=2
13	Planning Time: 2.814 ms
14	Execution Time: 0.523 ms

1. Identify Phases:

Bảng được Hashed: Là bảng language. Trình tối ưu hóa đã chọn bảng này để băm vì nó rất nhỏ. Nó quét qua bảng language và đưa vào node Hash để xây dựng bảng băm trên RAM.

Bảng được Probed: Là bảng film. Node này nằm bên ngoài, hệ thống sẽ quét toàn bộ 1000 dòng phim, cầm giá trị language_id của từng dòng đi dò vào bảng băm vừa tạo để tìm bản ghi khớp.

2. Memory Check:

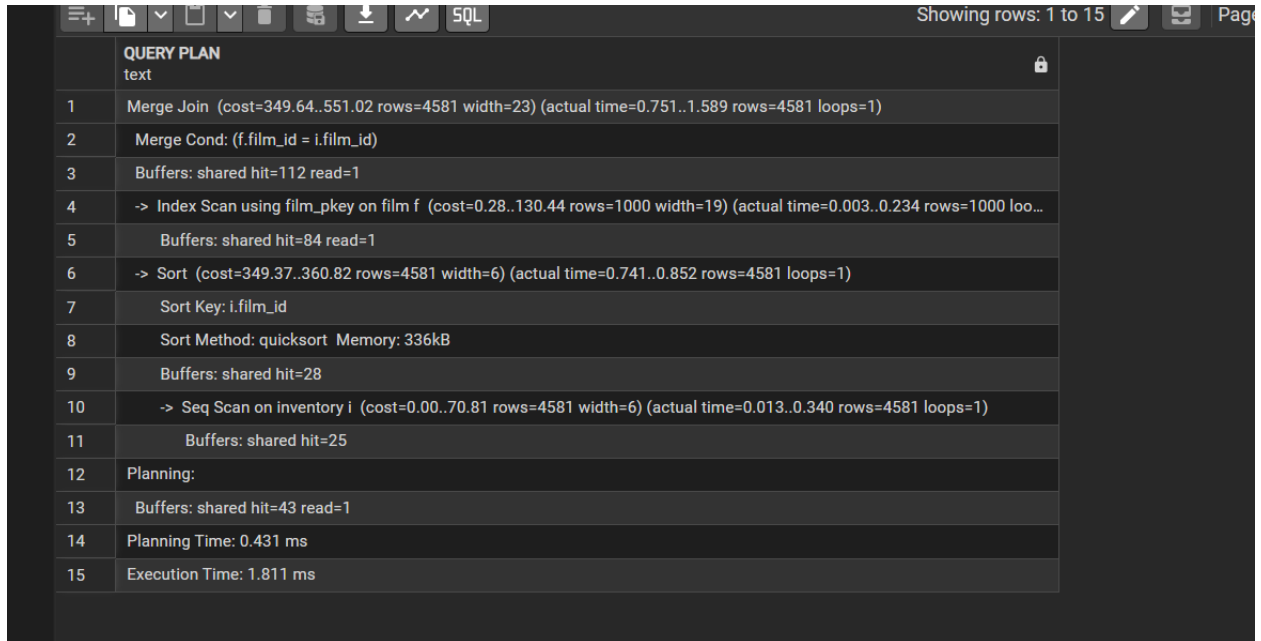
Nhìn vào Dòng 7 (Buckets: 1024 Batches: 1 Memory Usage: 9kB), dung lượng RAM được sử dụng để lưu trữ toàn bộ Hash Table chỉ tốn vốn vẹn 9kB.

3. Hypothesis:

Kết quả: Sẽ không có sự suy giảm hiệu năng hay tràn đĩa nào xảy ra. Mọi thứ vẫn giữ nguyên là Batches: 1.

Giải thích: Vì tổng dung lượng bảng băm của chúng ta chỉ tốn có 9kB. Kể cả khi ta giảm giới hạn RAM của hệ thống xuống mức tối thiểu là 64kB, thì 9kB vẫn nằm bên trong giới hạn đó. Trừ khi ta thực hiện gộp những bảng lớn khiến Hash Table phình to ra hàng Megabytes, thì lúc đó hệ thống mới bắt buộc phải chia thành nhiều mẻ (Batches > 1) và ghi tạm xuống ổ cứng, làm tốc độ chậm đi đáng kể.

Lab3:



	QUERY PLAN
	text
1	Merge Join (cost=349.64..551.02 rows=4581 width=23) (actual time=0.751..1.589 rows=4581 loops=1)
2	Merge Cond: (f.film_id = i.film_id)
3	Buffers: shared hit=112 read=1
4	-> Index Scan using film_pkey on film f (cost=0.28..130.44 rows=1000 width=19) (actual time=0.003..0.234 rows=1000 loo...
5	Buffers: shared hit=84 read=1
6	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=0.741..0.852 rows=4581 loops=1)
7	Sort Key: i.film_id
8	Sort Method: quicksort Memory: 336kB
9	Buffers: shared hit=28
10	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.013..0.340 rows=4581 loops=1)
11	Buffers: shared hit=25
12	Planning:
13	Buffers: shared hit=43 read=1
14	Planning Time: 0.431 ms
15	Execution Time: 1.811 ms

1.Structure:

Hệ thống đã sử dụng **kết hợp cả hai cách**.

Thứ nhất, ta có nhìn thấy một node Sort (ở dòng số 6). Tuy nhiên, nó chỉ áp dụng cho bảng inventory (sau khi bảng này bị quét bằng Seq Scan ở dòng 10). Hệ thống phải tốn 336kB RAM để chạy thuật toán quicksort sắp xếp bảng này.

Thứ hai, đối với bảng film, hệ thống không cần Sort mà lấy dữ liệu sắp xếp thông qua Index Scan. Vì cây B-Tree của khóa chính đã sắp xếp sẵn dữ liệu, hệ thống chỉ việc lấy dùng luôn.

2.Compare:

	QUERY PLAN	
	text	
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.282..1.050 rows=4581 loops=1)	
2	Hash Cond: (i.film_id = f.film_id)	
3	Buffers: shared hit=113	
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.007..0.183 rows=4581 loo...	
5	Buffers: shared hit=25	
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.265..0.265 rows=1000 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB	
8	Buffers: shared hit=88	
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.008..0.177 rows=1000 loo...	
10	Buffers: shared hit=88	
11	Planning:	
12	Buffers: shared hit=6	
13	Planning Time: 0.259 ms	
14	Execution Time: 1.164 ms	

Giải thích: Trình tối ưu hóa của Postgres luôn tính toán chi phí. Nhìn vào ảnh, để làm được Merge Join, nó phải quét toàn bộ 4581 dòng của bảng inventory rồi đưa vào bộ nhớ để chạy thuật toán băm. Chi phí cho việc ép buộc sắp xếp này cộng với chi phí đọc từ Index Scan của bảng film đắt hơn rất nhiều so với việc chỉ đơn giản là cho một bảng vào RAM tạo Hash Table . Vì vậy, nếu không bị ép buộc, nó luôn ưu tiên thuật toán rẻ nhất là Hash Join.